

Namespace e Projects

Modulo: 2 — Navigazione e Strumenti

Versione: OpenShift 4.14+ su Azure ARO

Obiettivi di Apprendimento

- Comprendere la differenza tra Namespace Kubernetes e Project OpenShift
- Creare, gestire ed eliminare Project con `oc new-project` e dalla console web
- Configurare ResourceQuota e LimitRange per limitare le risorse di un progetto
- Assegnare ruoli RBAC agli utenti a livello di progetto
- Usare label e annotazioni per organizzare e filtrare i progetti

1. Namespace vs Project: Qual è la Differenza?

In OpenShift un **Project** è un **Namespace Kubernetes esteso** con funzionalità aggiuntive:

Caratteristica	Namespace (Kubernetes)	Project (OpenShift)
Isolamento risorse	✓	✓
RBAC	✓ Manuale	✓ + Default RoleBindings automatici
Quote risorse	✓	✓
Annotazioni descrittive	Opzionali	✓ <code>display-name</code> , <code>description</code>
Visibilità nella console	API only	✓ Console Web integrata
Richiesta di creazione	<code>kubectl create ns</code>	<code>oc new-project</code> (con policy self-service)
Self-provisioning	✗	✓ (configurabile per utenti normali)

```
# Un Project è anche un Namespace – i comandi sono equivalenti:
oc get projects
oc get namespaces    # stesso risultato
```

```
# Ma 'oc new-project' fa più cose di 'kubectl create namespace':
# - crea il namespace
# - imposta annotazioni (display-name, description)
# - assegna automaticamente l'utente come admin del progetto
```

2. Creare e Gestire Project

2.1 Creazione da CLI

```
# Creazione base
oc new-project my-project

# Output atteso:
# Now using project "my-project" on server "https://api.cluster:6443".
# You can add applications to this project with the 'new-app' command.

# Creazione con metadati
oc new-project my-project \
  --display-name="La Mia Applicazione" \
  --description="Ambiente di sviluppo per il progetto MyApp"

# Verificare il progetto creato
oc get project my-project -o yaml
# Output (estratto):
# apiVersion: project.openshift.io/v1
# kind: Project
# metadata:
#   annotations:
#     openshift.io/description: Ambiente di sviluppo per il progetto MyApp
#     openshift.io/display-name: La Mia Applicazione
#   name: my-project
```

2.2 Navigare tra Project

```
# Visualizzare tutti i progetti accessibili
oc projects
# Output atteso:
# You have access to the following projects and can switch between them with 'oc
project <projectname>':
#
#   * default
#   kube-system
#   my-project (La Mia Applicazione)
#   openshift-monitoring
#
# Using project "my-project" on server "https://api.cluster:6443".

# Passare a un progetto specifico
oc project my-project

# Verificare il progetto corrente
oc project
# Output: Using project "my-project" on server "https://api.cluster:6443".
```

2.3 Eliminazione di un Project

```
# Eliminare un progetto (e TUTTE le risorse al suo interno)
oc delete project my-project

# Output atteso:
# project.project.openshift.io "my-project" deleted

# Attendere che il namespace sia completamente terminato
oc get namespace my-project
# Output atteso dopo qualche secondo:
# Error from server (NotFound): namespaces "my-project" not found
```

⚠ **Attenzione:** `oc delete project` elimina **irreversibilmente** tutte le risorse nel progetto (Pod, PVC, ConfigMap, Secret, ecc.). Verificare sempre prima di procedere.

3. ResourceQuota — Limitare le Risorse del Progetto

Una **ResourceQuota** impone limiti aggregati alle risorse consumabili all'interno di un namespace.

Tipi di Risorse Limitabili

Risorsa	Descrizione
<code>requests.cpu</code>	Somma delle CPU richieste da tutti i Pod
<code>requests.memory</code>	Somma della memoria richiesta da tutti i Pod
<code>limits.cpu</code>	Somma dei limiti CPU di tutti i Pod
<code>limits.memory</code>	Somma dei limiti memoria di tutti i Pod
<code>pods</code>	Numero massimo di Pod
<code>services</code>	Numero massimo di Service
<code>persistentvolumeclaims</code>	Numero massimo di PVC
<code>requests.storage</code>	Somma totale dello storage richiesto
<code>configmaps</code>	Numero massimo di ConfigMap
<code>secrets</code>	Numero massimo di Secret

Creare una ResourceQuota

```
# file: quota-my-project.yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: quota-my-project
  namespace: my-project
spec:
  hard:
    requests.cpu: "4"           # massimo 4 CPU totali richieste
    requests.memory: 8Gi        # massimo 8 GiB memoria richiesta
    limits.cpu: "8"             # massimo 8 CPU totali come limite
    limits.memory: 16Gi         # massimo 16 GiB memoria come limite
    pods: "20"                  # massimo 20 Pod
    services: "10"              # massimo 10 Service
    persistentvolumeclaims: "5" # massimo 5 PVC
    requests.storage: 50Gi      # massimo 50 GiB di storage totale
```

```
# Applicare la quota
oc apply -f quota-my-project.yaml

# Verificare le quote del progetto
oc get resourcequota -n my-project
# Output atteso:
# NAME                AGE    REQUEST                                     LIMIT
# quota-my-project    1m     pods: 2/20, requests.cpu: 500m/4, ...    limits.cpu:
# 1/8, ...

# Descrizione dettagliata con utilizzo corrente
oc describe resourcequota quota-my-project -n my-project
# Output:
# Name:                quota-my-project
# Namespace:           my-project
# Resource              Used    Hard
# -----
# limits.cpu            1      8
# limits.memory         2Gi    16Gi
# persistentvolumeclaims 1      5
# pods                  2      20
# requests.cpu          500m   4
# requests.memory       1Gi    8Gi
# requests.storage      10Gi   50Gi
# services               1      10
```

4. LimitRange — Valori Default e Limiti per Container

Un **LimitRange** impone vincoli e valori default a **singoli container e Pod**, a differenza della ResourceQuota che opera a livello aggregato.

```
# file: limitrange-my-project.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: limits-my-project
  namespace: my-project
spec:
  limits:
    # Limiti per singolo Container
    - type: Container
      default:           # limite applicato se non specificato nel Pod
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:    # request applicata se non specificata nel Pod
        cpu: "100m"
        memory: "128Mi"
      max:               # valore massimo consentito
        cpu: "2"
        memory: "2Gi"
      min:               # valore minimo consentito
        cpu: "50m"
        memory: "64Mi"
    # Limiti per singolo Pod
    - type: Pod
      max:
        cpu: "4"
        memory: "4Gi"
    # Limiti per PVC
    - type: PersistentVolumeClaim
      max:
        storage: "20Gi"
      min:
        storage: "1Gi"
```

```
# Applicare il LimitRange
oc apply -f limitrange-my-project.yaml
# Verificare
oc get limitrange -n my-project
oc describe limitrange limits-my-project -n my-project
```

⚠ Attenzione: Se è presente un LimitRange con `defaultRequest.cpu` e si deploya un Pod senza specificare `resources`, i valori default vengono applicati automaticamente. Questo è il comportamento **corretto e consigliato** per garantire lo scheduling.

5. RBAC a Livello di Progetto

Ruoli Predefiniti per Progetto

OpenShift definisce ruoli standard assegnabili a livello di namespace:

Ruolo	Descrizione
admin	Controllo completo del progetto (incluso RBAC)
edit	Creazione e modifica di risorse (no RBAC, no quota)
view	Sola lettura di tutte le risorse
basic-user	Accesso minimo: può vedere il progetto

Assegnare Ruoli a Utenti

```
# Assegnare il ruolo 'admin' a un utente nel progetto
oc adm policy add-role-to-user admin alice -n my-project
# Output: clusterrole.rbac.authorization.k8s.io/admin added: "alice"

# Assegnare il ruolo 'edit' a un gruppo
oc adm policy add-role-to-group edit dev-team -n my-project

# Assegnare il ruolo 'view' a un ServiceAccount
oc adm policy add-role-to-user view \
  system:serviceaccount:monitoring:prometheus -n my-project

# Rimuovere un ruolo
oc adm policy remove-role-from-user edit alice -n my-project

# Verificare i RoleBinding nel progetto
oc get rolebinding -n my-project
# Output atteso:
# NAME                                ROLE                                AGE
# admin                               ClusterRole/admin                   5m
# edit                                ClusterRole/edit                    2m
# system:deployers                     ClusterRole/system:deployer         5m
# system:image-builders                 ClusterRole/system:image-builder    5m
# system:image-pullers                  ClusterRole/system:image-puller     5m

# Verificare cosa può fare un utente nel progetto
oc auth can-i --list --as=alice -n my-project
oc auth can-i create pods --as=alice -n my-project
```

6. Label e Annotazioni sui Project

Le **label** permettono di organizzare e selezionare i progetti, utili per policy di rete, monitoring e automazione.

```
# Aggiungere label a un progetto
oc label namespace my-project \
  environment=development \
  team=backend \
  cost-center=IT-001

# Visualizzare i progetti filtrati per label
oc get projects -l environment=development
# Output:
# NAME           DISPLAY NAME           STATUS
# my-project     La Mia Applicazione    Active
# dev-frontend   Frontend Dev           Active

# Aggiungere annotazioni
oc annotate namespace my-project \
  contact-email=team@company.com \
  documentation-url=https://wiki.company.com/my-project

# Rimuovere una label
oc label namespace my-project cost-center-

# Visualizzare le label di un namespace
oc get namespace my-project --show-labels
```

Uso delle Label con NetworkPolicy

```
# Le label sui namespace permettono di scrivere NetworkPolicy basate sul progetto
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-monitoring
  namespace: my-project
spec:
  podSelector: {}
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: openshift-monitoring # label automatica
OCP
```

7. Self-Provisioning dei Project

Per default, tutti gli utenti autenticati in OCP possono creare nuovi Project (**self-provisioning**).

```
# Verificare se il self-provisioning è abilitato
oc describe clusterrolebinding self-provisioners

# Disabilitare il self-provisioning (solo amministratori creano progetti)
oc patch clusterrolebinding self-provisioners \
  -p '{"subjects": null}'

# Riabilitare il self-provisioning
oc adm policy add-cluster-role-to-group \
  self-provisioner \
  system:authenticated:oauth
```

8. Note Specifiche per Azure ARO

```
# Su ARO, i namespace con prefisso 'openshift-' e 'kube-' sono riservati e
# gestiti dal servizio – non modificarli

# Visualizzare i namespace di sistema (non modificabili su ARO)
oc get namespaces | grep -E "^openshift-|^kube-"

# Su ARO il namespace 'default' NON deve essere usato per workload applicativi:
# creare sempre Project dedicati

# Verificare le ResourceQuota di default applicate automaticamente da ARO
oc get resourcequota --all-namespaces | grep -v openshift | grep -v kube
```

 **Attenzione ARO:** Non eliminare o modificare i namespace con prefisso **openshift-** o **kube-**. Sono gestiti da Red Hat/Microsoft e la loro modifica può compromettere il funzionamento del cluster.

9. Esercizi Pratici

Esercizio 1 — Creare un Progetto con Quote

Obiettivo: creare un progetto con ResourceQuota e LimitRange.

```
# 1. Creare il progetto
oc new-project lab-quota \
  --display-name="Lab Quota" \
  --description="Progetto di test per ResourceQuota"

# 2. Applicare una ResourceQuota
cat <<EOF | oc apply -f -
apiVersion: v1
kind: ResourceQuota
metadata:
  name: lab-quota
  namespace: lab-quota
spec:
  hard:
    pods: "5"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
EOF

# 3. Applicare un LimitRange
cat <<EOF | oc apply -f -
apiVersion: v1
kind: LimitRange
metadata:
  name: lab-limits
  namespace: lab-quota
spec:
  limits:
  - type: Container
    default:
      cpu: "200m"
      memory: "256Mi"
    defaultRequest:
      cpu: "100m"
      memory: "128Mi"
EOF

# 4. Tentare di creare un Pod che supera i limiti
cat <<EOF | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: test-over-quota
  namespace: lab-quota
```

```
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal
    command: [sleep, "3600"]
    resources:
      requests:
        cpu: "800m"      # questo Pod + altri supererebbe la quota di 1 CPU
        memory: "800Mi"
EOF
# → Osservare il messaggio di errore se la quota è già parzialmente utilizzata

# 5. Verificare lo stato della quota
oc describe resourcequota lab-quota -n lab-quota
```

Esercizio 2 — Gestione RBAC del Progetto

Obiettivo: assegnare e verificare ruoli nel progetto.

```
# 1. Creare un progetto
oc new-project lab-rbac

# 2. Visualizzare i RoleBinding creati automaticamente
oc get rolebinding -n lab-rbac

# 3. Assegnare il ruolo 'view' all'utente corrente in un secondo progetto
oc new-project lab-readonly
oc adm policy add-role-to-user view $(oc whoami) -n lab-rbac

# 4. Verificare i permessi
oc auth can-i get pods -n lab-rbac --as=$(oc whoami)      # → yes
oc auth can-i create pods -n lab-rbac --as=$(oc whoami)  # → no (view only)
oc auth can-i delete pods -n lab-rbac --as=$(oc whoami)  # → no (view only)

# 5. Promuovere a 'edit'
oc adm policy add-role-to-user edit $(oc whoami) -n lab-rbac
oc auth can-i create pods -n lab-rbac --as=$(oc whoami)  # → yes ora

# 6. Pulizia
oc delete project lab-rbac lab-readonly lab-quota
```

10. Riepilogo dei Punti Chiave

- Un **Project** OCP è un Namespace Kubernetes con metadati aggiuntivi, default RoleBindings e gestione self-provisioning
 - `oc new-project` crea il namespace, imposta le annotazioni e assegna automaticamente il creatore come `admin`
 - La **ResourceQuota** limita le risorse aggregate nel progetto; il **LimitRange** impone valori default e limiti ai singoli container
 - Il **RBAC a livello di progetto** usa i ruoli `admin`, `edit`, `view` assegnabili con `oc adm policy add-role-to-user`
 - Le **label** sui namespace sono essenziali per NetworkPolicy, monitoring e automazione basata su selettori
 - Su **ARO** non modificare i namespace `openshift-*` e `kube-*`: sono gestiti dal servizio
-

Riferimenti

- [Working with Projects - Red Hat](#)
- [Resource Quotas - Red Hat](#)
- [LimitRange - Red Hat](#)
- [RBAC in OpenShift](#)